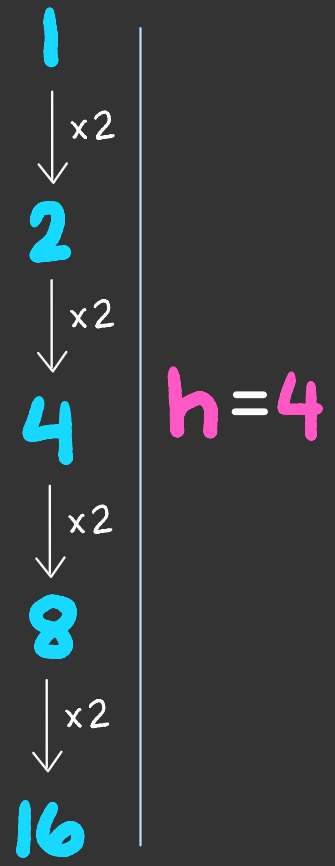
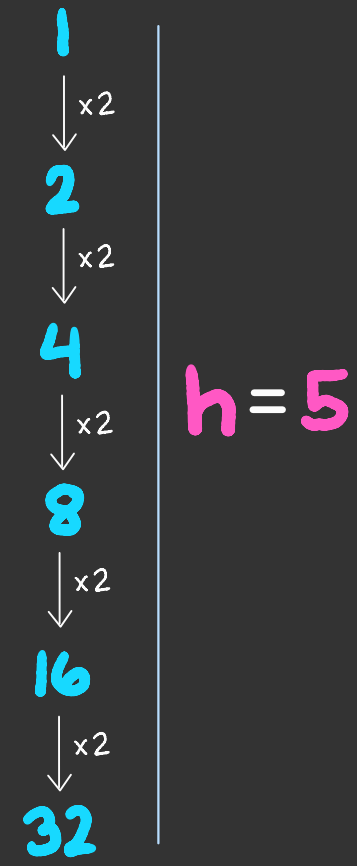
logarithmic complexity

CSCI 134

if we start with the number 1 and double it h times, what do we get?

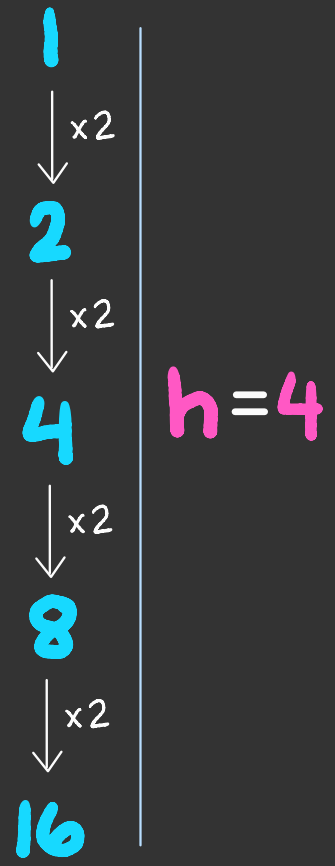
?

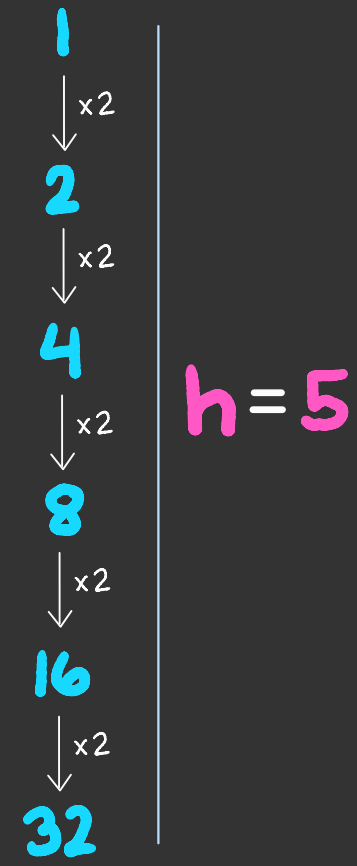




if we start with the number 1 and double it h times, what do we get?

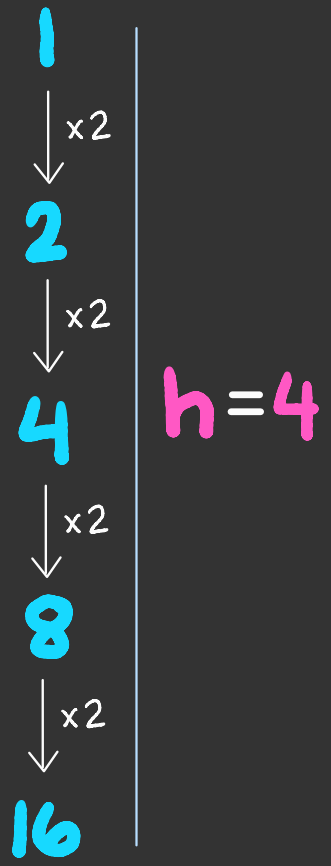
$$2^h$$

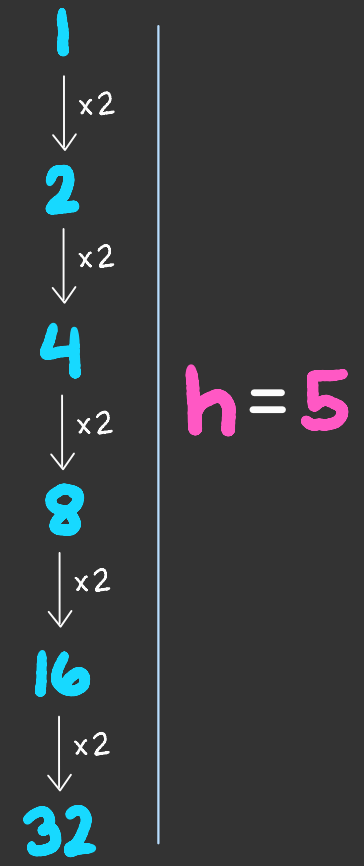




if we start with the number 1, how many times do we need to double it to reach n?

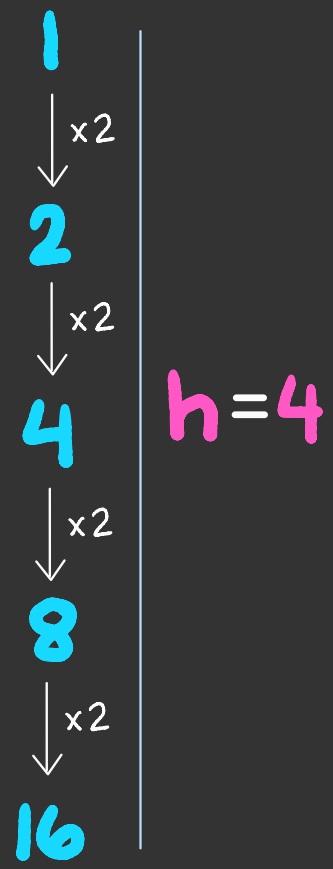
?

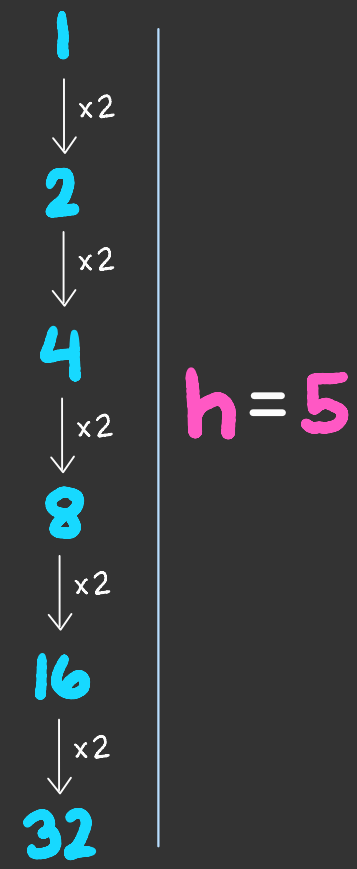




if we start with the number 1, how many times do we need to double it to reach n?

$$\log_2 n$$

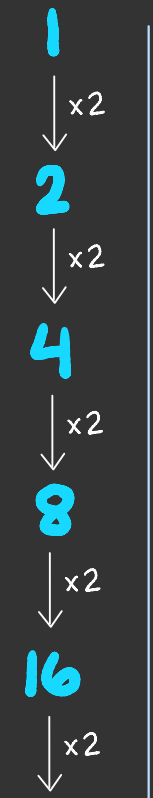




log

is the opposite of

pow



$\log_2 32 = 5$

$pow(2,5) = 32$

log

is the opposite of

pow

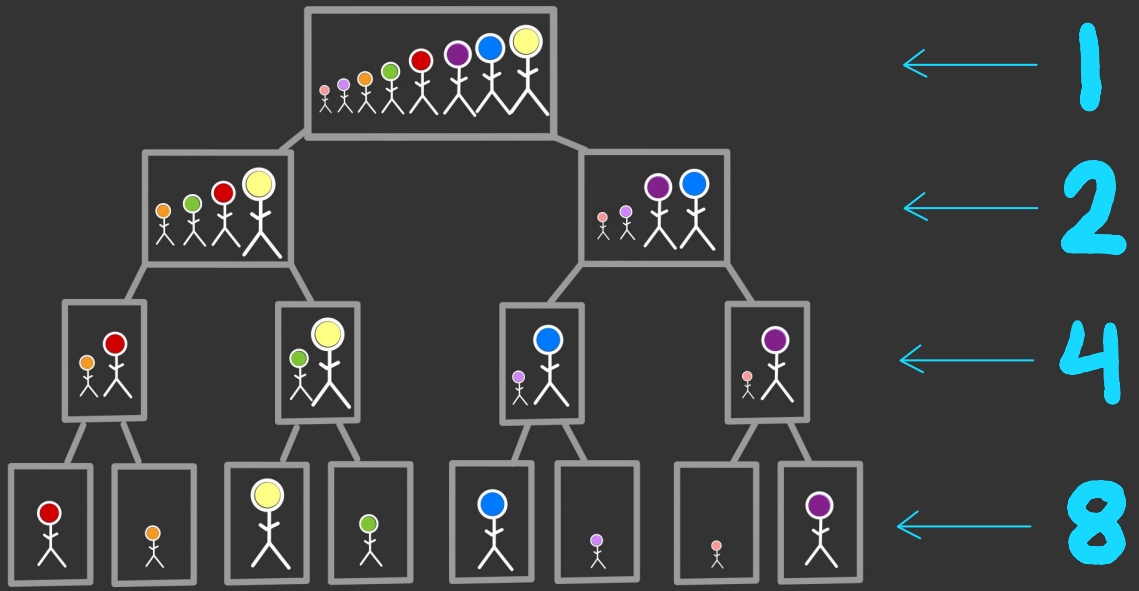
if:

$$2^h = n$$

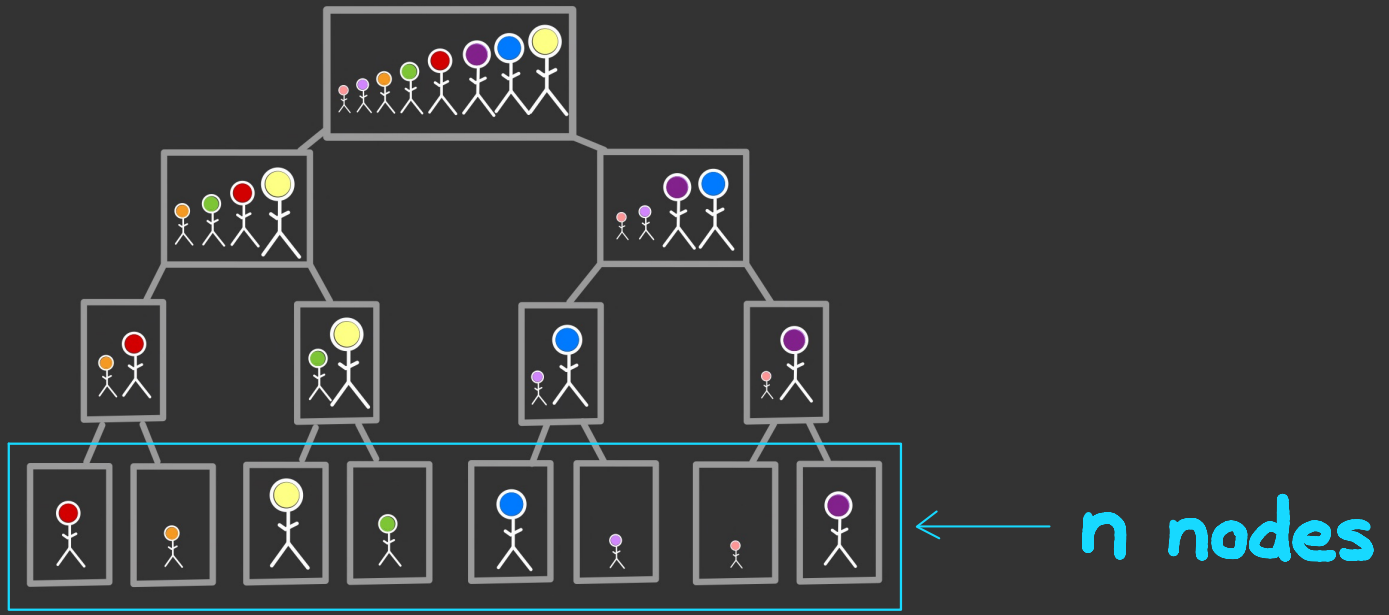
then:

$$\log_2 n = h$$

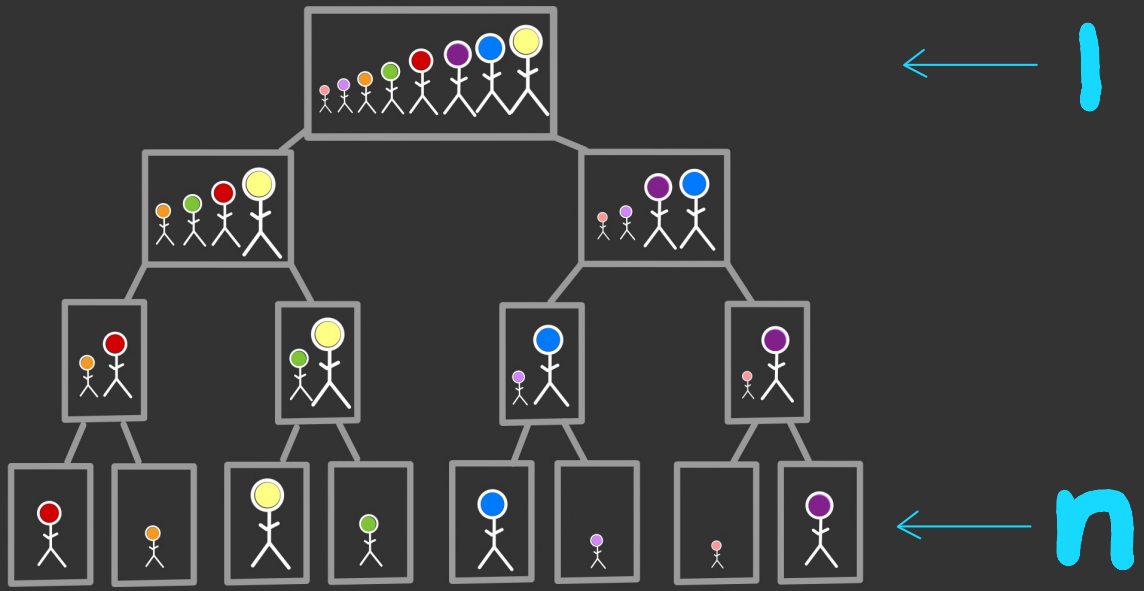
at each level of the tree, the number of nodes doubles



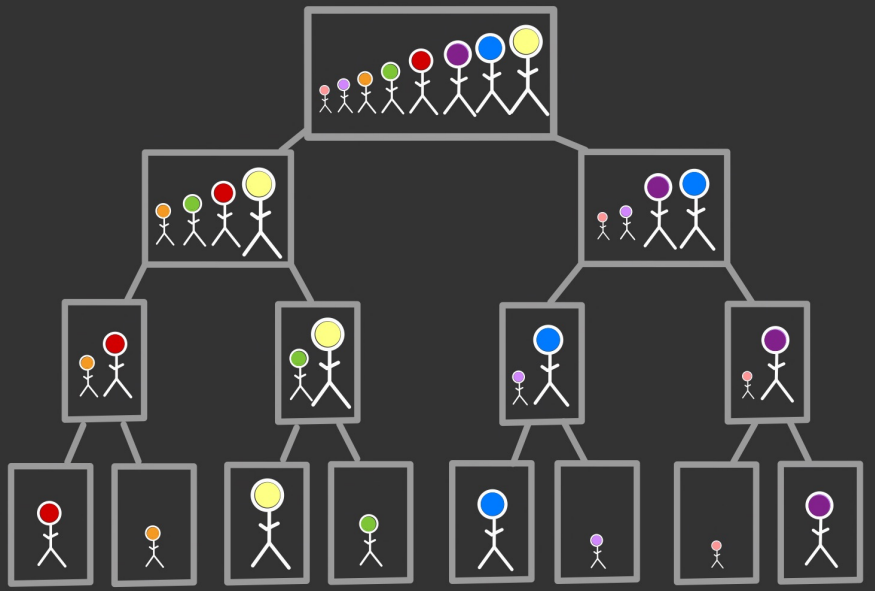
the final level has a node for each element in the list we want to sort



if we start with the number 1, how many times do we need to double it to reach n?



if we start with the number 1, how many times do we need to double it to reach n?

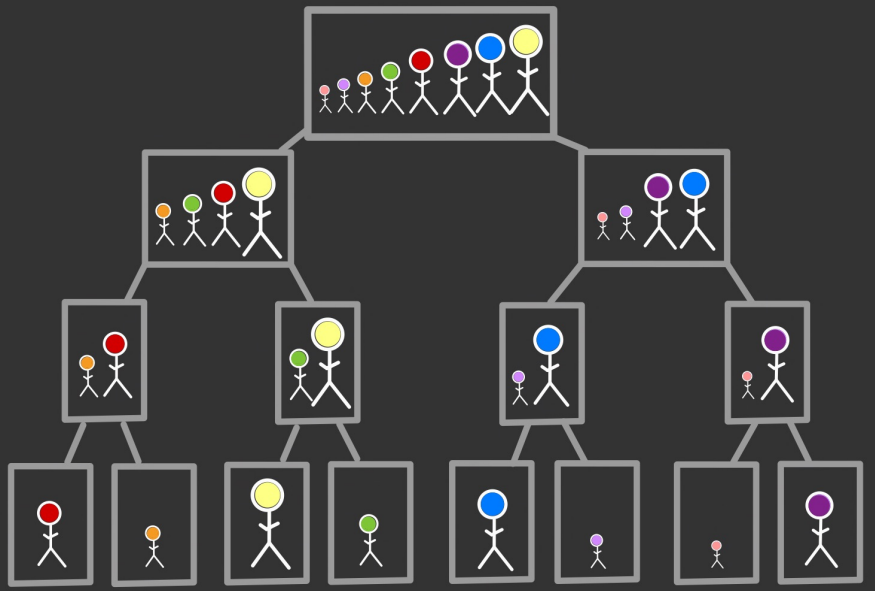


1

$h = \log_2 n$

n

so the height of the tree is $\log_2 n$



1

$h = \log_2 n$

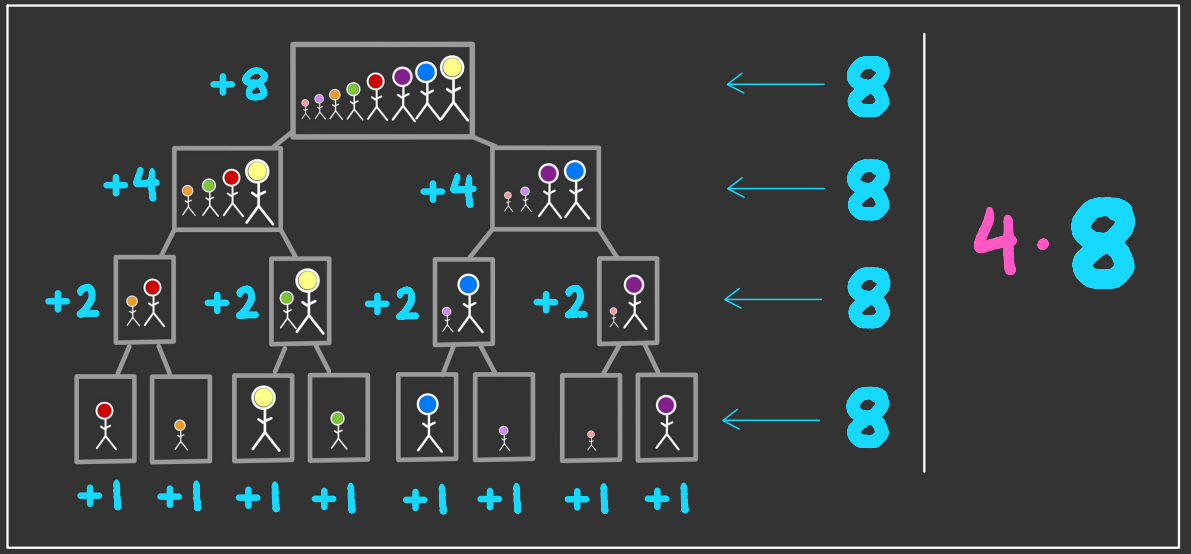
n

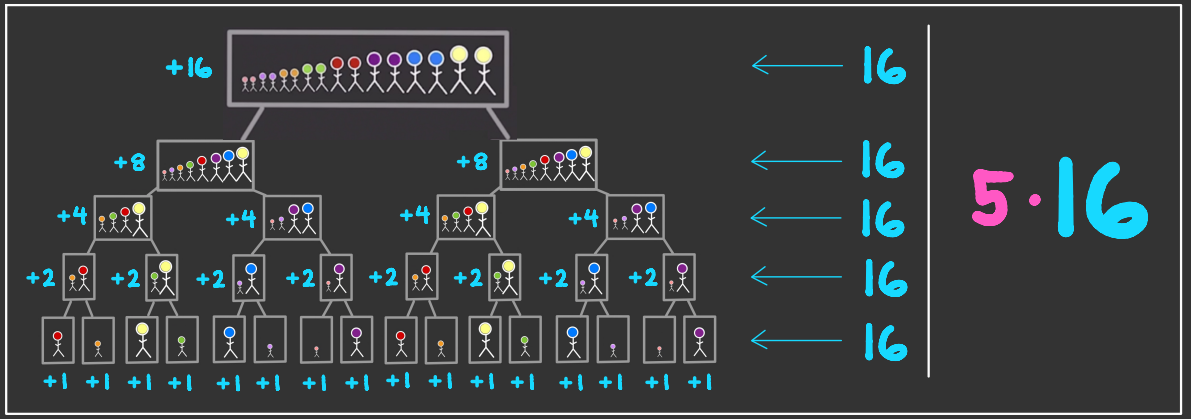
recall:

mergesort makes a total of

hn

comparisons, where h is the height of the tree



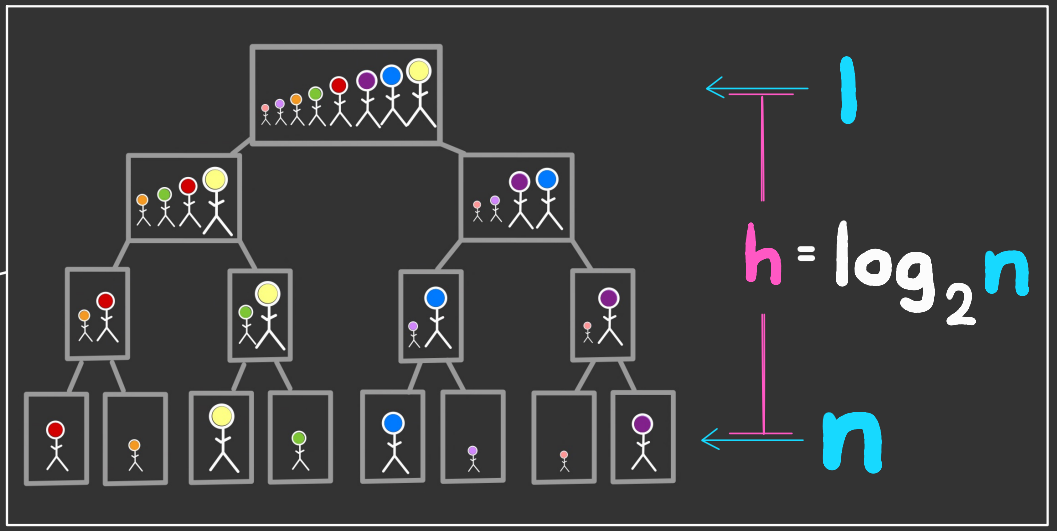


recall:

mergesort makes a total of

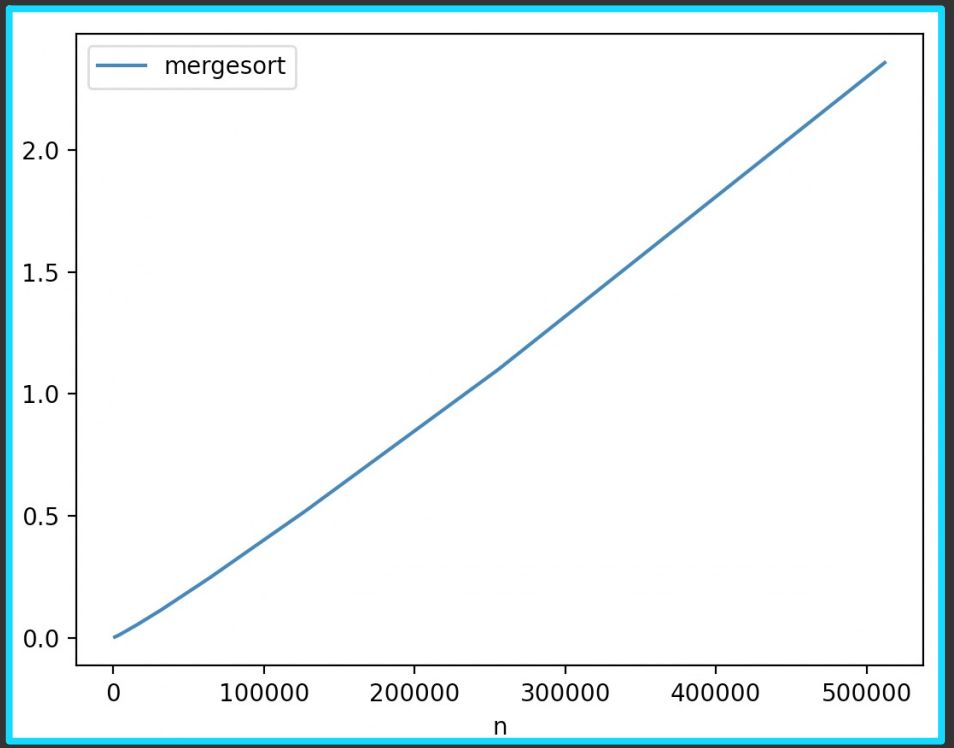
hn

comparisons, where h is the height of the tree

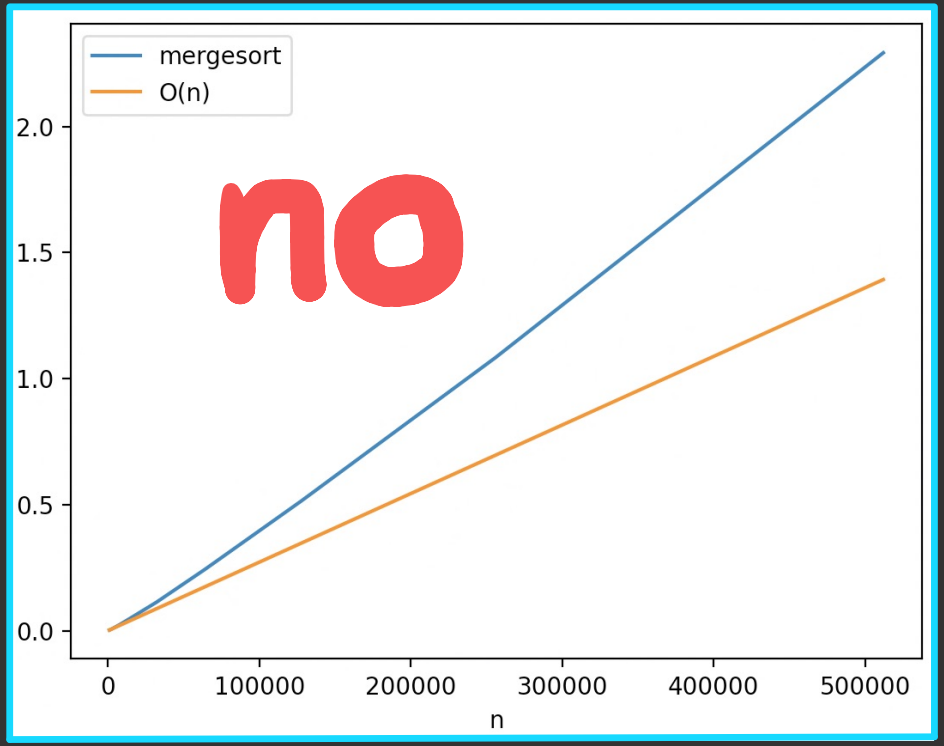


so mergesort makes $n \log_2 n$ comparisons

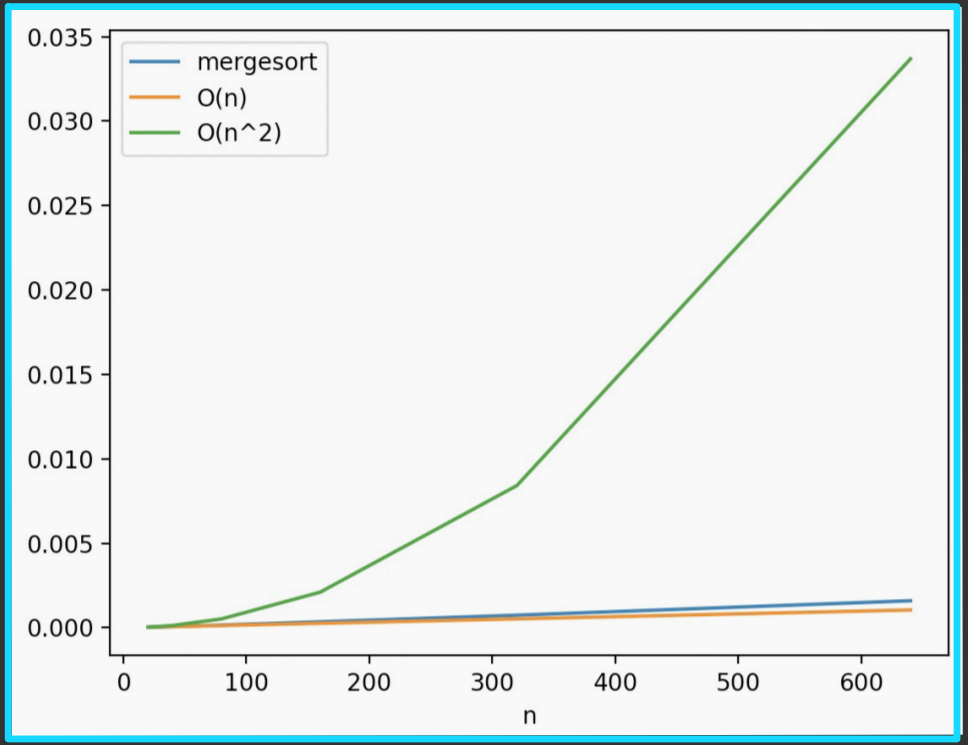
is $n \log_2 n \in O(n)$?



is $n \log_2 n \in O(n)$? ✗



but $n \log_2 n$ is way better than $O(n^2)$



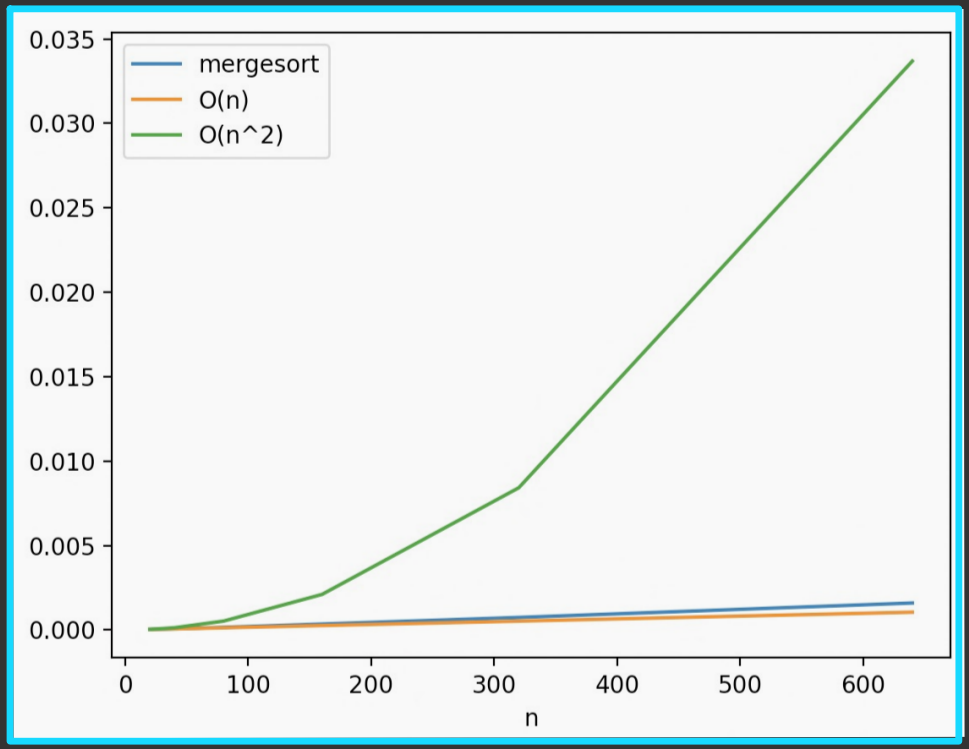
$O(n)$

is better than

$O(n \log_2 n)$

is better than

$O(n^2)$



$O(n)$

is better than

$O(n \log_2 n)$

is better than

$O(n^2)$

turns out this doesn't matter

$n \log_a n \in O(n \log_b n)$

for any positive numbers

a and b

$O(n)$

is better than

$O(n \log_{\square} n)$

is better than

$O(n^2)$

so typically the base is omitted when discussing logarithmic complexity

common complexity classes

slower

$O(1)$	"constant time"	e.g. list indexing
$O(\log n)$	"logarithmic time"	e.g. binary search
$O(n)$	"linear time"	e.g. linear search
$O(n \log n)$	"quasilinear time"	e.g. mergesort
$O(n^2)$	"quadratic time"	e.g. bubble sort